

UNIT II: Relational Model

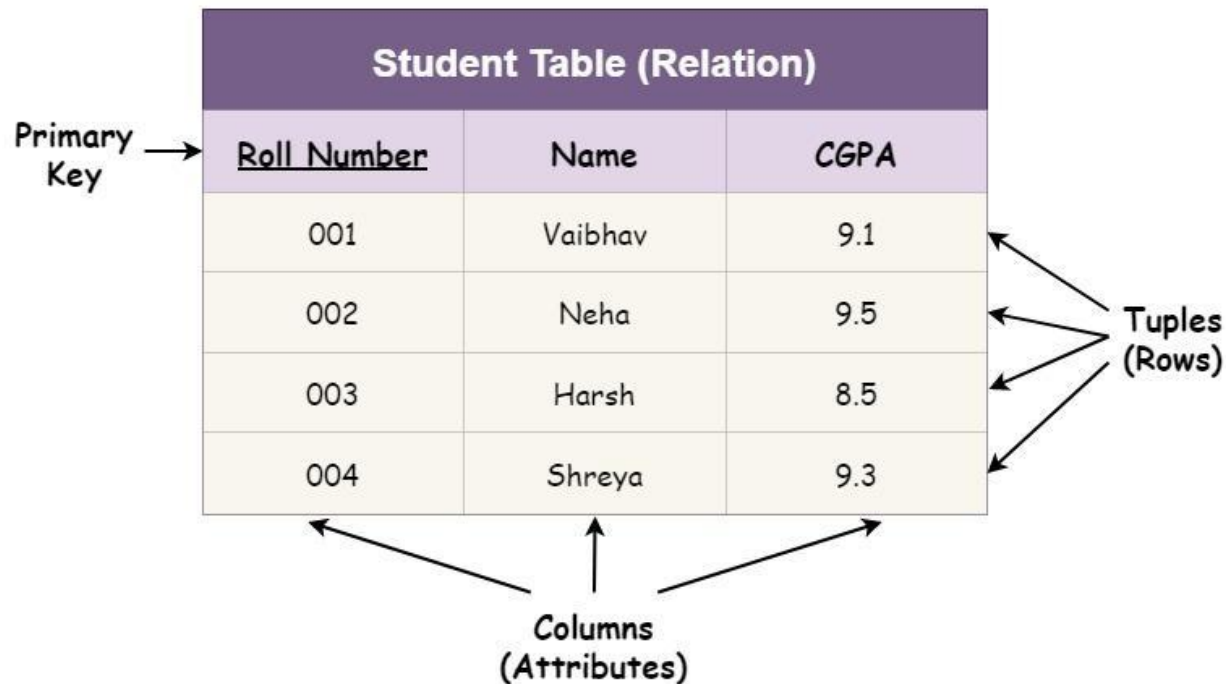
- Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus.
- BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

UNIT II: Relational Model

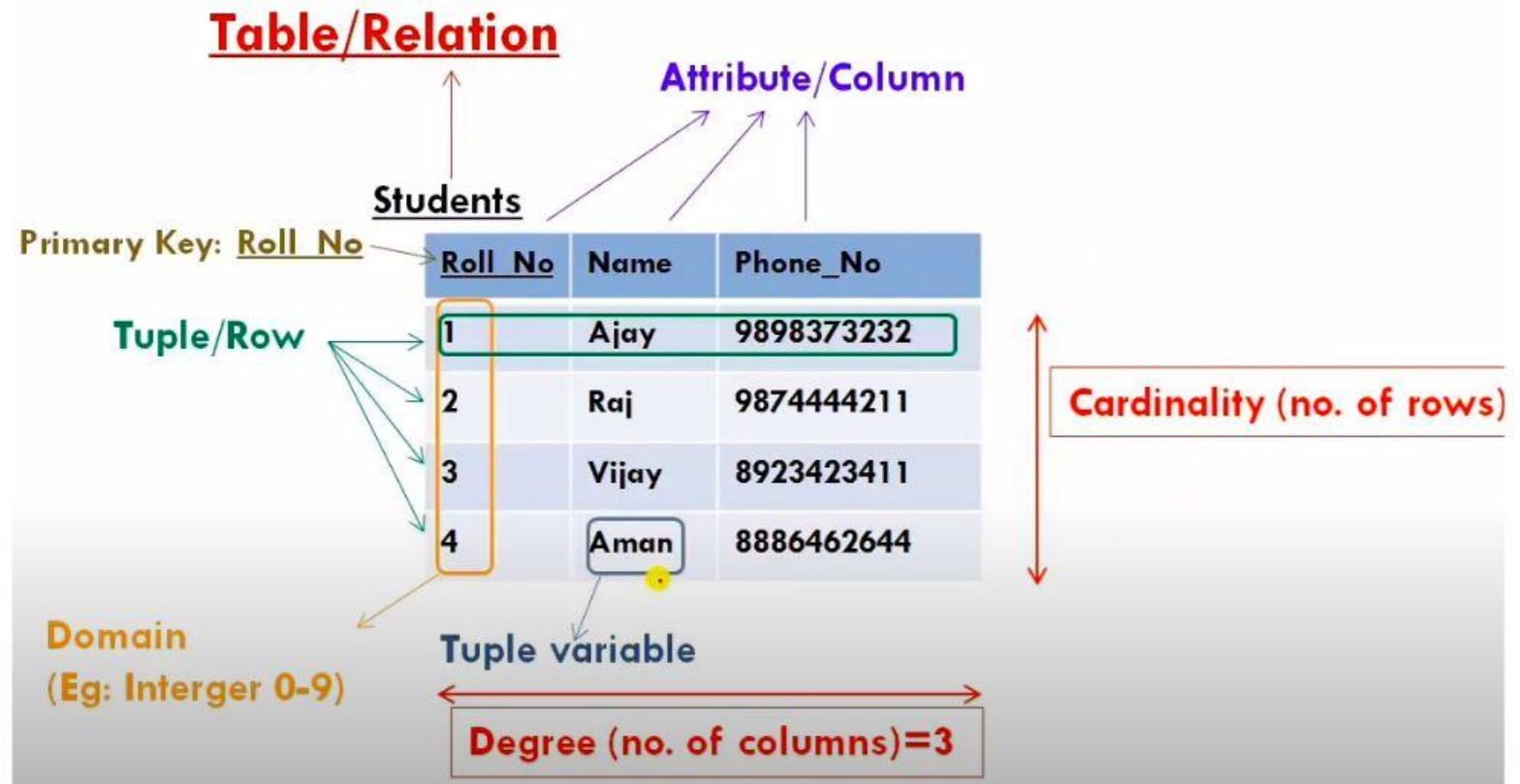
- Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus.
- BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

Introduction to relational model

Relational Model in DBMS



Introduction to relational model



Introduction to relational model

- The Relational Model organizes data using **tables (relations)** consisting of rows and columns.
- The relational model **represents how data is stored and managed** in Relational Databases where data is organized into tables, each known as a relation.
- Each **row of a table represents an entity or record** and each **column represents a particular attribute** of that entity.
- The relational model transforms conceptual designs from **ER diagrams into implementable structures**.
- These structures are used in relational database systems like **Oracle SQL and MySQL**.

Introduction to relational model

- Relational Model Concepts in DBMS :
 - **Attribute:** Each column in a Table. Attributes are the properties which define a relation. e.g., Student_Rollno, Name, etc.
 - **Tables:** In the Relational model the, relations are saved in the table format. It is stored along with its entities.
 - **Tuple:** It is nothing but a single row of a table, which contains a single record.
 - **Relation Schema:** A relation schema represents the name of the relation with its attributes.

Introduction to relational model

- **Degree:** The total number of attributes which in the relation is called the degree of the relation.
- **Cardinality:** Total number of rows present in the Table.
- **Attribute domain:** Every attribute has some pre-defined value and scope which is known as attribute domain

UNIT II: Relational Model

- Introduction to relational model, concepts of domain, attribute, tuple, relation, **importance of null values**, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus.
- BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

Importance of null values

- ❑ NULL is a condition.
- ❑ NULL is not the number zero.
- ❑ NULL is not the empty string("") value.
- ❑ SQL NULL is the term used to represent a missing value.
- ❑ Rather, NULL is the value used to represent an unknown piece of data.
- ❑ Remember: a NULL is an unknown value in which the field appears blank.

Importance of null values

This is not a NULL Value

StudentID	StudentName	City	marks
1	Raj		45
2	Sham	Mumbai	NULL
3	Tom	Pune	66
4	Ram	Pune	58
5	Joy	Mumbai	NULL
6	Robin	NULL	NULL

NULL VALUE

Importance of null values

- Syntax: The basic syntax of **NULL** while creating a table.

```
SQL> CREATE TABLE CUSTOMERS(  
    ID      INT                NOT NULL,  
    NAME VARCHAR (20)          NOT NULL,  
    AGE     INT                NOT NULL,  
    ADDRESS CHAR (25) ,  
    SALARY  DECIMAL (18, 2),  
    PRIMARY KEY (ID)  
);
```

- There are two columns where we did not use NOTNULL, which means these **columns could be NULL**

Importance of null values

```
CREATE TABLE Employee (  
    Fname VARCHAR(50),  
    Lname VARCHAR(50),  
    SSN VARCHAR(11),  
    Phoneno VARCHAR(15),  
    Salary FLOAT  
);  
  
INSERT INTO Employee (Fname, Lname, SSN, Phoneno, Salary)  
VALUES  
    ('Shubham', 'Thakur', '123-45-6789', '9876543210', 50000.00),  
    ('Aman', 'Chopra', '234-56-7890', NULL, 45000.00),  
    ('Aditya', 'Arpan', NULL, '8765432109', 55000.00),  
    ('Naveen', 'Patnaik', '345-67-8901', NULL, NULL),  
    ('Nishant', 'Jain', '456-78-9012', '7654321098', 60000.00);  
  
Select * FROM Employee;
```

Importance of null values

Fname	Lname	SSN	Phoneno	Salary
Shubham	Thakur	123-45-6789	9876543210	50000
Aman	Chopra	234-56-7890		45000
Aditya	Arpan		8765432109	55000
Naveen	Patnaik	345-67-8901		
Nishant	Jain	456-78-9012	7654321098	60000

Importance of null values

- **The IS NULL Operator:**

- SELECT Fname, Lname FROM Employee WHERE SSN IS NULL;

- **The IS NOT NULL Operator:**

- SELECT COUNT(*) AS Count FROM Employee WHERE SSN IS NOT NULL;

- **Updating NULL Values in a Table:**

- UPDATE Employee
SET SSN = '789-01-2345'
WHERE Fname = 'Aditya' AND Lname = 'Arpan';

Importance of null values

Fname	Lname	SSN	Phoneno	Salary
Shubham	Thakur	123-45-6789	9876543210	50000
Aman	Chopra	234-56-7890		45000
Aditya	Arpan	789-01-2345	8765432109	55000
Naveen	Patnaik	345-67-8901		
Nishant	Jain	456-78-9012	7654321098	60000

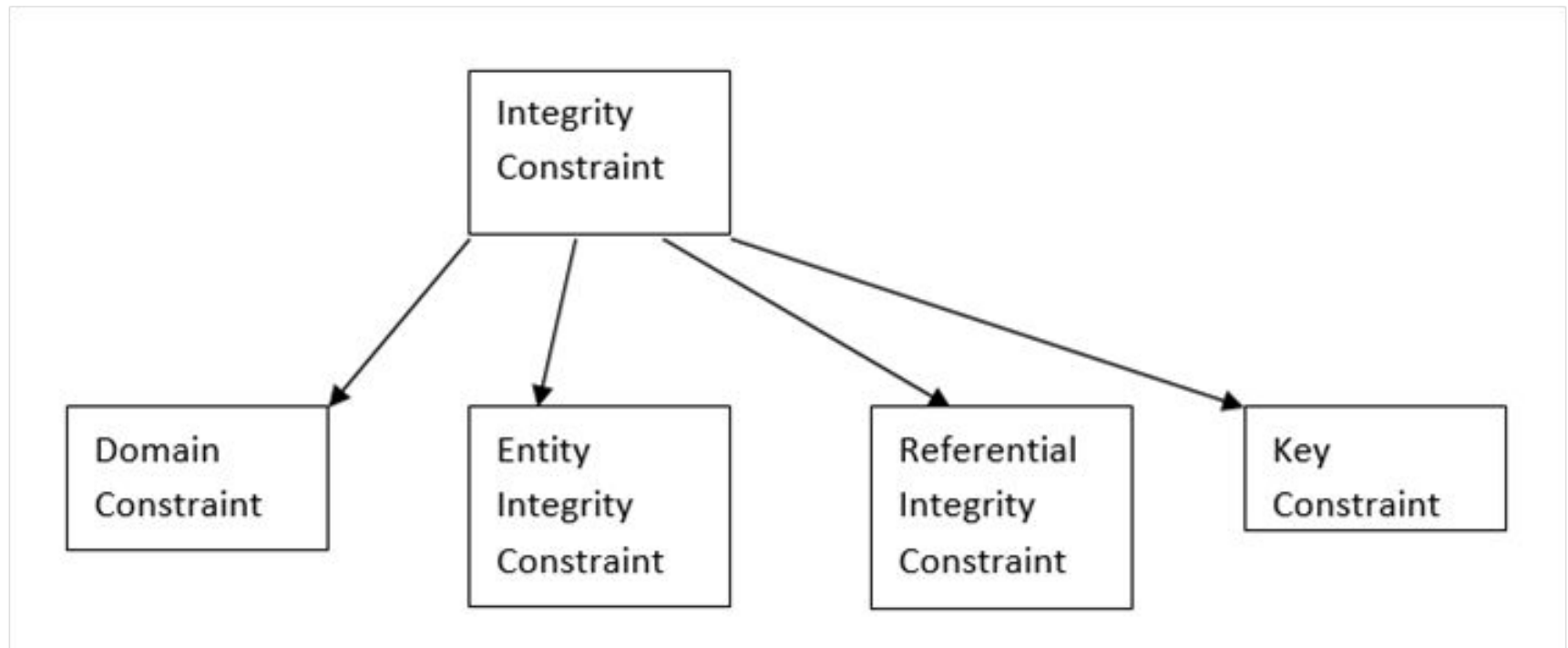
UNIT II: Relational Model

- Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus.
- BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

Constraints (Domain, Key constraints, integrity constraints)

- In DBMS (Database Management Systems), constraints are rules enforced on data within a database to **ensure accuracy, consistency, and integrity**.
- They **act as safeguards, preventing invalid data from being entered** and maintaining relationships between data elements.
- Constraints can be divided into following two types:
 - **Column level constraints:** limits to only column data
 - **Table level constraints:** limits to whole table data

Constraints (Domain, Key constraints, integrity constraints)



Constraints (Domain, Key constraints, integrity constraints)

- Domain Constraints ensure the integrity and validity of data within a database by restricting the values that can be stored in a column.
- Essentially, Domain Constraints define the permissible values for a given attribute, ensuring that the data adheres to specific rules and standards.
- They prevent invalid data from being entered into the database.

Constraints (Domain, Key constraints, integrity constraints)

- For example, we can specify if a particular column can hold **null values or not**, if the values have to be **unique or not**, the **data type or size of values** that can be entered in the column, the **default values** for the column, etc.
- Domain Constraint = data type(integer / character / date / time / string / etc.) + Constraints(NOT NULL / UNIQUE / PRIMARY KEY / FOREIGN KEY / CHECK / DEFAULT)

Constraints (Domain, Key constraints, integrity constraints)

- Not Null
- Unique
- Default
- Check
- Key Constraints (Super Key, Candidate Key, Primary Key, Alternate Key, Foreign Key)
- Integrity Constraints / Referential Integrity Constraints

Constraints (Domain, Key constraints, integrity constraints)

- Not Null
- Unique
- Default
- Check
- Key Constraints (Super Key, Candidate Key, Primary Key, Alternate Key, Foreign Key)
- Integrity Constraints / Referential Integrity Constraints

Not Null

- ❑ CREATE TABLE Persons (
 ID int NOT NULL,
 LastName varchar(255) NOT NULL,
 FirstName varchar(255) NOT NULL,
 Age int
);
- ❑ Desc Students;

Not Null

```
SQL> desc Persons
```

Name	Null?	Type
-----	-----	-----
ID	NOT NULL	NUMBER(38)
LASTNAME	NOT NULL	VARCHAR2(255)
FIRSTNAME	NOT NULL	VARCHAR2(255)
AGE		NUMBER(38)

Not Null

- ❑ insert into Persons values ('','A', 'B ', 20);
- ❑ insert into Persons values (,'A', 'B ', 20);
- ❑ insert into Persons values (NULL,'A', 'B ', 20);

Not Null

```
SQL> insert into Persons values (,'A', 'B ', 20)
```

ORA-00936: missing expression

<https://docs.oracle.com/error-help/db/ora-00936/>

Error at Line: 4 Column: 0

```
SQL> insert into Persons values ('','A', 'B ', 20)
```

ORA-01400: cannot insert NULL into ("SQL_N7N5Z6QPVVU8WFLQZWI0QKE32G"."PERSONS"."ID")

<https://docs.oracle.com/error-help/db/ora-01400/>

Error at Line: 4 Column: 0

Not Null

```
SQL> insert into Persons values (02,'A', 'B ',NULL )
```

```
1 row inserted.
```

```
Elapsed: 00:00:00.003
```

Not Null

- Select * from Persons;

	ID	LASTNAME	FIRSTNAME	AGE
1	1	ABC	XYZ	20
2	2	A	B	20 (null)

Constraints (Domain, Key constraints, integrity constraints)

- Not Null
- Unique
- Default
- Check
- Key Constraints (Super Key, Candidate Key, Primary Key, Alternate Key, Foreign Key)
- Integrity Constraints / Referential Integrity Constraints

Unique

- ❑ CREATE TABLE Persons1 (
 ID int NOT NULL,
 LastName varchar(255) NOT NULL,
 FirstName varchar(255) UNIQUE,
 Age int
);
- ❑ Desc Persons1;

Unique

□ `select * from Persons1;`



Download ▼

Execution time: 0.012 seconds

	ID	LASTNAME	FIRSTNAME	AGE
	2	BCD	YYY	20
!	1	ABC	XYZ	20

Unique

- insert into Persons1 values (02, 'ABC' , 'XYZ' , 21);

ORA-

00001: unique constraint (SQL_N7N5Z6QPVVU8WFLQZWI0QKE32G.SYS_C002879826) violated on table SQL_N7N5Z6QPVVU8WFL
ORA-03301: (ORA-00001 details) row with column values (FIRSTNAME:'XYZ') already exists

<https://docs.oracle.com/error-help/db/ora-00001/>

Error at Line: 4 Column: 0

Constraints (Domain, Key constraints, integrity constraints)

- Not Null
- Unique
- **Default**
- Check
- Key Constraints (Super Key, Candidate Key, Primary Key, Alternate Key, Foreign Key)
- Integrity Constraints / Referential Integrity Constraints

Default

- ❑ CREATE TABLE Persons3 (
 ID int NOT NULL,
 LastName varchar(255) NOT NULL,
 FirstName varchar(255),
 Age int,
 City varchar(255) DEFAULT 'Vijayawada'
- ❑);
- ❑ Desc Persons3;

Default

- insert into Persons3 values (01, 'ABC' , 'XYZ' , 20, 'Bhimavaram');
- insert into Persons3 values (01, 'BCD' , 'YZZ' , 21, 'Vizag');

	ID	LASTNAME	FIRSTNAME	AGE	CITY
1	1	BCD	YZZ	21	Vizag
2	1	ABC	XYZ	20	Bhimavarai

Default

- insert into Persons3 values (05, 'BBB' , 'ZZZ' , 20, DEFAULT);
- Select * from Persons3;

	ID	LASTNAME	FIRSTNAME	AGE	CITY
1	1	BCD	YZZ	21	Vizag
2	1	ABC	XYZ	20	Bhimavarai
3	5	BBB	ZZZ	20	Vijayawada

Constraints (Domain, Key constraints, integrity constraints)

- Not Null
- Unique
- Default
- Check
- Key Constraints (Super Key, Candidate Key, Primary Key, Alternate Key, Foreign Key)
- Integrity Constraints / Referential Integrity Constraints

Check

- ❑ CREATE TABLE Persons2 (
ID int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Age int CHECK (Age >= 18)
- ❑);
- ❑ Desc Persons2;

Check

- Select * from Persons2;

	ID	LASTNAME	FIRSTNAME	AGE
1	1	ABC	XYZ	20
2	2	AAA	YYZ	20

Check

- insert into Persons2 values (03, 'BCC' , 'XYZ' , 17);

```
SQL> insert into Persons2 values (03, 'BCC' , 'XYZ' , 17)
```

```
ORA-02290: check constraint (SQL_N7N5Z6QPVVU8WFLQZWI0QKE32G.SYS_C002879936) violated
```

```
https://docs.oracle.com/error-help/db/ora-02290/
```

```
Error at Line: 4 Column: 0
```

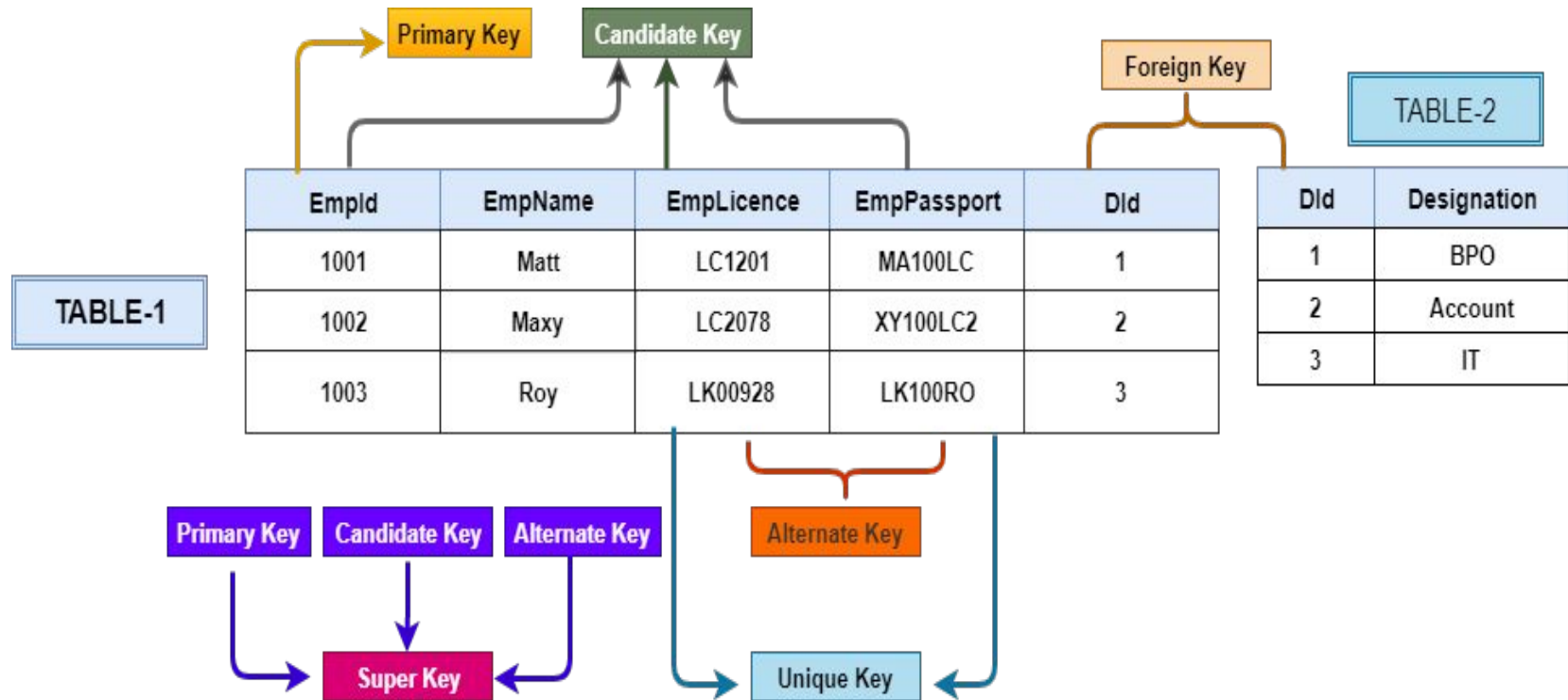
Constraints (Domain, Key constraints, integrity constraints)

- Not Null
- Unique
- Default
- Check
- Key Constraints (Super Key, Candidate Key, Primary Key, Alternate Key, Foreign Key)
- Integrity Constraints / Referential Integrity Constraints

Constraints (Domain, Key constraints, integrity constraints)

- Super Key
- Candidate Key
- Primary Key
- Alternate Key
- Foreign Key or Referencing Key

Constraints (Domain, Key constraints, integrity constraints)



Constraints (Domain, Key constraints, integrity constraints)

- **Super Key:** The set of attributes that can uniquely identify a tuple is known as Super Key. For Example, STUD_NO, Mobile_no, Pancard, (STUD_NO, STUD_NAME), (Mobile_no, Name), etc.
- **Candidate Key:** The minimal set of attributes that can uniquely identify a tuple is known as a candidate key. For Example, STUD_NO in STUDENT relation.

Constraints (Domain, Key constraints, integrity constraints)

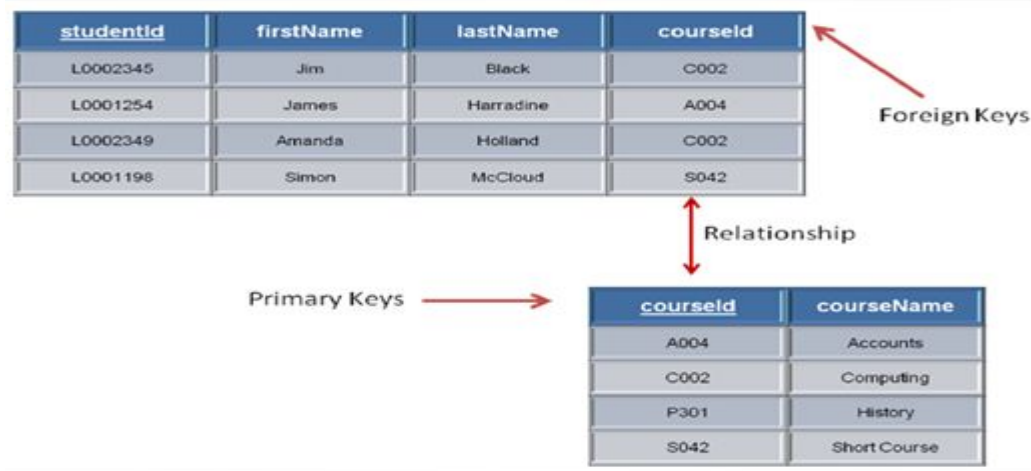
- **Primary Key:** There can be more than one candidate key in relation out of which one can be chosen as the primary key. For Example, STUD_NO, as well as STUD_PHONE, are candidate keys for relation STUDENT but STUD_NO can be chosen as the primary key.
- **Alternate Key:** The candidate key other than the primary key is called an alternate key.

Constraints (Domain, Key constraints, integrity constraints)

- A foreign key is a key used to **link two tables together**.
- This is sometimes also called as a **referencing key**.
- A Foreign Key is a column or a combination of columns whose **values match a Primary Key in a different table**.
- Referential Integrity Constraint ensures that there always exists a **valid relationship between two tables**.
- The relationship between 2 tables matches the Primary Key in one of the tables with a Foreign Key in the second table.

Constraints (Domain, Key constraints, integrity constraints)

- An example might be a student table that contains the **course_id** the student is attending. Another table lists the courses on offer with **course_id** being the primary key.
- The 2 tables are **linked through course_id** and as such **course_id** would be a foreign key in the student table.



Constraints (Domain, Key constraints, integrity constraints)

- **Primary Key Example:**
- ```
CREATE TABLE Students (
 RegNo INT PRIMARY KEY,
 FirstName VARCHAR(50),
 LastName VARCHAR(50),
 Mobile INT
);
```

# Constraints (Domain, Key constraints, integrity constraints)

## □ Desc Students;

```
Table STUDENTS created.
```

```
Elapsed: 00:00:00.017
```

---

```
SQL> Desc Students
```

| Name      | Null?    | Type         |
|-----------|----------|--------------|
| -----     | -----    | -----        |
| REGNO     | NOT NULL | NUMBER(38)   |
| FIRSTNAME |          | VARCHAR2(50) |
| LASTNAME  |          | VARCHAR2(50) |
| MOBILE    |          | NUMBER(38)   |

# Constraints (Domain, Key constraints, integrity constraints)

- insert into Students (RegNo, FirstName, LastName, Mobile) Values (1,'AA', 'BB',123),  
(2,'CC','DD',456),  
(3,'EE','FF',789),  
(4,'GG','HH',890);

# Constraints (Domain, Key constraints, integrity constraints)

|   | REGNO | FIRSTNAME | LASTNAME | MOBILE |
|---|-------|-----------|----------|--------|
| 1 | 1     | AA        | BB       | 123    |
| 2 | 2     | CC        | DD       | 456    |
| 3 | 3     | EE        | FF       | 789    |
| 4 | 4     | GG        | HH       | 890    |

# Constraints (Domain, Key constraints, integrity constraints)

- **Foreign Key Example:**
- ```
CREATE TABLE AadharId1 (  
    AadharNo INT PRIMARY KEY,  
    RegNo INT,  
    District Varchar(10),  
    FOREIGN KEY (RegNo) REFERENCES  
    Students(RegNo)  
);
```

Constraints (Domain, Key constraints, integrity constraints)

- Insert into AadharId1 Values
- (112233,1,'Kri'),
- (223344,2,'EG'),
- (334455,3,'WG'),
- (445566,4,'VSKP');

Constraints (Domain, Key constraints, integrity constraints)

	AADHARNO	REGNO	DISTRICT
1	112233	1	Kri
2	223344	2	EG
3	334455	3	WG
4	445566	4	VSKP

Constraints (Domain, Key constraints, integrity constraints)

- `select S.RegNo,S.Mobile,A.AadharNo,A.District`
- `from Students S`
- `Join AadharID1 A ON S.RegNo=A.RegNo;`

Constraints (Domain, Key constraints, integrity constraints)

- select S.RegNo,S.Mobile,A.AadharNo,A.District
- from Students S
- Join AadharID1 A ON S.RegNo=A.RegNo;

Constraints (Domain, Key constraints, integrity constraints)

	REGNO	MOBILE	AADHARNO	DISTRICT
1	1	123	112233	Kri
2	2	456	223344	EG
3	3	789	334455	WG
4	4	890	445566	VSKP

UNIT II: Relational Model

- Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, **Relational Algebra**, Relational Calculus.
- BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

Relational Algebra

- ▣ Developed in the Year 1970 by EF Codd (Father of DBMS)
- ▣ Uses Mathematical Expressions
- ▣ Theoretical Model
- ▣ Base for SQL
- ▣ Selection and Projection ,Set Operations, Renaming Joins, Division.

Relational Algebra-Selection

- The SELECT operation is used for selecting a subset of the tuples according to a given selection condition.
- **Sigma(σ)**-Symbol denotes it.
- Select operator **selects tuples that satisfy a given predicate.**
- Syntax: $\sigma_p(r)$
- σ is the predicate, r stands for relation which is the name of the table, p is propositional logic formula which may use connectors like and, or, and not. These terms may use relational operators like $=, \neq, \geq, <, >, \leq$.

Relational Algebra-Selection

- $\sigma_{\text{regdno}}(\text{Student})$
- $\sigma_{\text{regdno,name}}(\text{Student})$
- $\sigma_{\text{regdno}=1}(\text{Student})$

Relational Algebra-Projection

- The projection method defines a relation that contains a **vertical subset of Relation**.
- This helps to **extract the values of specified attributes to eliminates duplicate values**.
- **(pi)** symbol is used to choose attributes from a relation.
- This operator helps you to keep specific columns from a relation and discards the other columns.

Relational Algebra-Projection

CustomerID	CustomerName	Status
1	Google	Active
2	Amazon	Active
3	Apple	Inactive
4	Alibaba	Active

Relational Algebra-Projection

- $\Pi_{\text{CustomerName, Status}}(\text{Customers})$

CustomerName	Status
Google	Active
Amazon	Active
Apple	Inactive
Alibaba	Active

Relational Algebra-Selection, Projection

- Select name from Student where Regdno=2;

Regd no	Name	Age
1	A	20
2	B	21
3	A	19

Relational Algebra-Selection, Projection

- Select name from Student where Regdno=2;

Regd no	Name	Age
1	A	20
2	B	21
3	A	19

- $\Pi_{\text{Name}} (\sigma_{\text{regdno}=2} (\text{Student}))$

- $\sigma_{\text{regdno}=2} (\Pi_{\text{name}} (\text{Student}))$

Relational Algebra-Set Operations

- Union
- Set Difference
- Intersection
- Cartesian Product / Cross Product

Relational Algebra- Union

- UNION is symbolized by $\cup$ symbol. It includes all tuples that are in tables A or in B. It also eliminates duplicate tuples.
- So, set A UNION set B would be expressed as:
 - $A \cup B$
- For a union operation to be valid, the following conditions must hold –
 - A and B must be the same number of attributes.
 - Attribute domains need to be compatible.
 - Duplicate tuples will be automatically removed.

Relational Algebra- Union

□ $A \cup B$

Table A	
column 1	column 2
1	1
1	2

Table B	
column 1	column 2
1	1
1	3

Relational Algebra- Union

□ $A \cup B$

Table $A \cup B$	
column 1	column 2
1	1
1	2
1	3

Relational Algebra- Set Difference

- $-$ Symbol denotes it.
- The result of $A - B$, is a relation which includes all tuples that are in A but not in B .
- For a set Difference operation to be valid, the following conditions must hold
 - The attribute name of A has to match with the attribute name in B .
 - The relations A and B should be union compatible.

Relational Algebra- Set Difference

□ A - B

Table A	
column 1	column 2
1	1
1	2

Table B	
column 1	column 2
1	1
1	3

Relational Algebra- Set Difference

□ $A - B$

Table A - B	
column 1	column 2
1	2

Relational Algebra- Intersection

- An intersection is defined by the symbol $\cap$
- Defines a relation consisting of a set of all tuples that are in both A and B.
- However, A and B must be union-compatible.

Relational Algebra- Intersection

□ $A \cap B$

Table A	
column 1	column 2
1	1
1	2

Table B	
column 1	column 2
1	1
1	3

Relational Algebra- Intersection

□ $A \cap B$

Table $A \cap B$	
column 1	column 2
1	1

Relational Algebra-Cartesian Product

- Cartesian Product in DBMS is an operation used to **merge columns from two relations**.
- Generally, a Cartesian product is **never a meaningful operation** when it performs alone.
- However, it becomes meaningful when it is followed by other operations (**Joins**).
- It is also called **Cross Product or Cross Join**.

Relational Algebra-Cartesian Product

□ Employee Table:

EMPLOYEE		
EMP_ID	EMP_NAME	EMP_DEPT
1	Smith	A
2	Harry	C
3	John	B

Relational Algebra-Cartesian Product

□ Department Table:

DEPARTMENT	
DEPT_NO	DEPT_NAME
A	Marketing
B	Sales
C	Legal

Relational Algebra-Cartesian Product

□ Employee X Department

EMP_ID	EMP_NAME	EMP_DEPT	DEPT_NO	DEPT_NAME
1	Smith	A	A	Marketing
1	Smith	A	B	Sales
1	Smith	A	C	Legal
2	Harry	C	A	Marketing
2	Harry	C	B	Sales
2	Harry	C	C	Legal
3	John	B	A	Marketing
3	John	B	B	Sales
3	John	B	C	Legal

Relational Algebra- Division

- As the name of this operation implies, it involves dividing one relation by another.
- Notation: r / s or $r \div s$
- Useful for expressing queries that include a “for all” or “for every” phrase

Relational Algebra- Division

Examples of Division A/B

A

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

B1

pno
p2

A/B1

sno
s1
s2
s3
s4

B2

pno
p2
p4

A/B2

sno
s1
s4

B3

pno
p1
p2
p4

A/B3

sno
s1

Relational Algebra-Rename

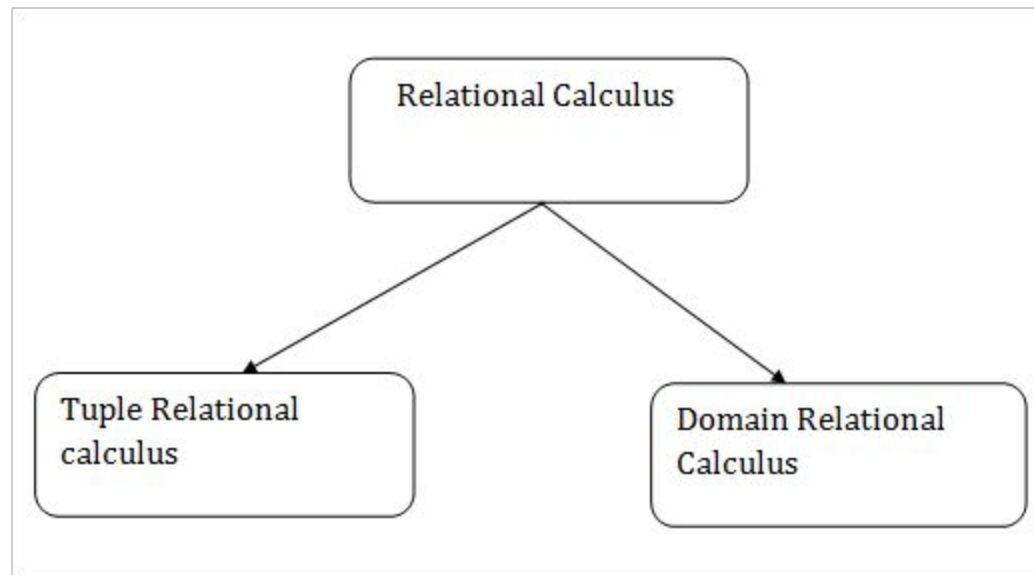
- The rename operation is used to **rename the output relation**.
- It is denoted by rho (ρ).
- $\rho(\text{STUDENT1}, \text{STUDENT})$

UNIT II: Relational Model

- Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, **Relational Calculus**.
- BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

Relational Calculus

- Tuple relational calculus which was originally proposed by Codd in the year 1972.
- Domain relational calculus which was proposed by Lacroix and Pirotte in the year 1977.



Relational Algebra vs Relational Calculus

Relational Algebra	Relational Calculus
Relational Algebra is procedural query language .	Relational Calculus is a non-procedural query language .
Relational Algebra targets how to obtain the result .	Relational Calculus targets what result to obtain .
Relational Algebra specifies the order in which operations are to be performed.	Relational Calculus specifies no such order of executions for its operations.
Relational Algebra is close to programming language concepts .	Relational Calculus is not related to programming language concepts .

Relational Calculus: Tuple

- Tuple relational calculus is used for **selecting those tuples that satisfy the given condition.**
- It describes the desired information **without giving a specific procedure** for obtaining that information.
- In TRC, filtering variable uses the tuples of a relation.
- The result of the relation can have one or more Tuples.

Relational Calculus: Tuple

Notation:

- A Query in the tuple relational calculus is expressed as following notation
- $\{T \mid P(T)\}$ or $\{T \mid \text{Condition}(T)\}$
- Where
 - **T** is the resulting tuples
 - **P(T)** is the condition used to fetch T.

Relational Calculus: Tuple

□ Employee Table:

First_Name	Last_Name	Age
Ajeet	Singh	30
Chaitanya	Singh	31
Rajeev	Bhatia	27
Kiran	Pratap	28

- $\{ t.\text{Last_Name} \mid \text{Student}(t) \text{ AND } t.\text{age} > 30 \}$
- $\{ t \mid \text{Student}(t) \text{ AND } t.\text{Last_Name} = \text{'Singh'} \}$

Relational Calculus: Domain

- The second form of relation is known as Domain relational calculus.
- In domain relational calculus, **filtering variable uses the domain of attributes.**
- It uses logical connectives $\wedge$ (and), $\vee$ (or) and $\neg$ (not).
- It uses Existential ($\exists$, there exists) and Universal Quantifiers ($\forall$, for all) to bind the variable.

Relational Calculus: Domain

- **Notation:**
- $\{ a_1, a_2, a_3, \dots, a_n \mid P(a_1, a_2, a_3, \dots, a_n) \}$
- Where
 - a_1, a_2 are attributes
 - P stands for formula built by inner attributes

Relational Calculus: Domain

Employee Table:

First_Name	Last_Name	Age
Ajeet	Singh	30
Chaitanya	Singh	31
Rajeev	Bhatia	27
Kiran	Pratap	28

- $\{ \langle \text{First_Name}, \text{Last_name}, \text{Age} \rangle \mid \in \text{Student} \wedge \text{Age} > 27 \}$
- $\{ \langle \text{First_Name}, \text{Age} \rangle \mid \in \text{Student} \wedge \text{Age} \geq 27 \}$

UNIT II: Relational Model

- Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus.
- BASIC SQL: **Simple Database schema**, data types, table definitions (create, alter), different DML operations (insert, delete, update).

Simple Database schema

- ****Explain this concept with the help of any table creation example from the lab experiments***

UNIT II: Relational Model

- Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus.
- BASIC SQL: Simple Database schema, **data types**, table definitions (create, alter), different DML operations (insert, delete, update).

Data types

Data Types in Structured Query Language (SQL)

Data Type	Description	Example
Numeric	Used to store numbers (both integers and decimals).	INT, DECIMAL, FLOAT
String	Stores alphanumeric values (textual data).	CHAR, VARCHAR, TEXT
Date & Time	Used to store date and time-related values.	DATE, TIME, DATETIME, TIMESTAMP
Boolean	Used to store true/false values.	BOOLEAN
Binary	Stores binary data like images, files, etc.	BLOB, BINARY, VARBINARY

UNIT II: Relational Model

- Introduction to relational model, concepts of domain, attribute, tuple, relation, importance of null values, constraints (Domain, Key constraints, integrity constraints) and their importance, Relational Algebra, Relational Calculus.
- BASIC SQL: Simple Database schema, data types, table definitions (create, alter), different DML operations (insert, delete, update).

table definitions (create, alter),
different DML operations (insert,
delete, update).

- DDL and DML Commands.